

Math & Statistics Foundations

Derivatives, the chain rule, gradients, probability, Bayes' theorem, softmax, and normalization

Concept Guide 2 of 9 | Read before: [any neural network or ML lab](#) | Companion to the [Zero-to-Hero series](#)

1. Derivatives: The Rate of Change

A derivative measures how fast a function's output changes as its input changes — the slope of the function at a point. Every model-training algorithm in this course uses derivatives to decide which direction to adjust a parameter.

FORMULA — The derivative

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

in practice you use known rules (power rule, chain rule) rather than the limit directly

WORKED EXAMPLE — Power rule example

$$f(x) = x^2 \rightarrow f'(x) = 2x$$

At $x=3$: $f(3)=9$, $f'(3)=6$ (the function is increasing at a rate of 6 units per unit of x)

At $x=0$: $f'(0)=0$ (a flat point — the minimum of x^2)

MENTAL MODEL

The derivative tells you which way is 'downhill' on a loss curve. If $f'(x)$ is positive, decreasing x reduces $f(x)$. If $f'(x)$ is negative, increasing x reduces $f(x)$. Gradient descent is nothing more than repeatedly stepping in the downhill direction.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.3 \(Autograd\)](#)

2. The Chain Rule

The chain rule lets you compute the derivative of a function built from nested functions — exactly the situation in a neural network, where the loss depends on the output, which depends on the last layer, which depends on the layer before it, and so on.

FORMULA — The chain rule

$$\frac{d}{dx} [f(g(x))] = f'(g(x)) * g'(x)$$

the derivative of a composition is the product of the derivatives of each piece

WORKED EXAMPLE — Worked example

Let $y = (2x + 1)^2$. Think of this as $f(u) = u^2$ where $u = g(x) = 2x + 1$.

$$f'(u) = 2u, g'(x) = 2$$

$$dy/dx = f'(g(x)) * g'(x) = 2 * (2x + 1) * 2 = 4 * (2x + 1)$$

$$\text{At } x=1: y=(3)^2=9, dy/dx = 4 * 3 = 12$$

DIAGRAM — Chain rule as a relay

$x \rightarrow [2x+1] \rightarrow u \rightarrow [u^2] \rightarrow y$

to get dy/dx : multiply the LOCAL derivatives along the chain

$$dy/dx = (dy/du) * (du/dx)$$

[PRACTICE THIS → 17_Neural_Networks_Deep_Dive/02_Backpropagation; Transformers_Attention_Zero_to_Hero.ipynb Ch.3](#)

3. Partial Derivatives and Gradients

A model has many parameters (weights), not just one. A partial derivative measures how the loss changes with respect to ONE parameter, holding all others fixed. The gradient is the vector of all these partial derivatives at once — it points in the direction of steepest increase.

FORMULA — The gradient vector

$$\text{grad}(f) = [df/dx_1, df/dx_2, \dots, df/dx_n]$$

one partial derivative per parameter, collected into a vector

MENTAL MODEL

Imagine standing on a hilly landscape where height = loss. The gradient at your feet points directly uphill. To minimize loss, you step in the OPPOSITE direction of the gradient — this is exactly what 'gradient descent' means by name.

COMMON MISUNDERSTANDINGS

- The gradient points toward INCREASING the function — that's why training subtracts the gradient (moves downhill), not adds it.
- A gradient of zero doesn't always mean you found the best solution — it could be a local minimum, a saddle point, or the true global minimum. Neural network loss surfaces have many such flat points.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.3-4](#)

4. Probability Basics

Probability quantifies uncertainty. Two ideas recur constantly in this course: how probabilities combine, and how to update a belief given new evidence (Bayes' theorem).

FORMULA — Bayes' theorem

$$P(A|B) = P(B|A) * P(A) / P(B)$$

the probability of A given B, in terms of the reverse conditional probability

WORKED EXAMPLE — A classic worked example

1% of people have a disease. A test is 99% accurate (for both positive and negative cases).

You test positive. What's $P(\text{disease} | \text{positive test})$?

$P(\text{pos}|\text{disease})=0.99$, $P(\text{disease})=0.01$, $P(\text{pos})= 0.99*0.01 + 0.01*0.99 = 0.0198$

$P(\text{disease}|\text{pos}) = 0.99*0.01 / 0.0198 = 0.50$ (only 50%! not 99%)

MENTAL MODEL

This surprising result is why base rates matter: even a highly accurate test can give a coin-flip result when the underlying condition is rare. This same logic underlies why classifier evaluation (precision/recall in the Guardrails lab) matters more than raw accuracy on imbalanced data.

PRACTICE THIS → [20_Math_and_Statistics_Foundations; Evaluation_Guardrails_Zero_to_Hero.ipynb Ch.4](#)

5. Softmax: Turning Numbers into Probabilities

Softmax converts a list of raw scores (logits) into a probability distribution that sums to 1. It appears in classification outputs, attention weights, and next-token prediction in language models.

FORMULA — Softmax

$$\text{softmax}(x)[i] = \exp(x[i]) / \sum(\exp(x[j])) \text{ for all } j$$

exponentiate everything, then normalize so it sums to 1

WORKED EXAMPLE — Worked example

logits = [2.0, 1.0, 0.1]

exp: [7.389, 2.718, 1.105] sum = 11.212

softmax: [0.659, 0.242, 0.099] (sums to 1.0)

The highest logit (2.0) gets the highest probability (0.659), but not ALL the probability.

COMMON MISUNDERSTANDINGS

- Without a numerical-stability trick (subtracting the max logit before exponentiating), large logits can overflow to infinity in floating point — every implementation in this course subtracts the max first.
- 'Temperature' scales the logits before softmax: dividing by a small number makes the distribution sharper (more confident); dividing by a large number flattens it (more random).

PRACTICE THIS → [Transformers_Attention_Zero_to_Hero.ipynb Ch.5;](#)

[LLM_APIs_Prompting_Zero_to_Hero.ipynb Ch.3](#)

6. Variance, Standard Deviation, and Normalization

Variance measures how spread out data is. Standardizing features (subtracting the mean, dividing by the standard deviation) is required before k-NN, k-Means, PCA, and SVMs — and is good practice before training most models.

FORMULA — Standardization (z-score)

$$z = (x - \text{mean}) / \text{std_dev}$$

after this transform, the feature has mean 0 and standard deviation 1

WORKED EXAMPLE — Why scale matters

Feature A: house size, ranges 500-3500 (spread of ~3000)

Feature B: bedrooms, ranges 1-5 (spread of ~4)

Without scaling, distance calculations (k-NN, k-Means) are dominated almost entirely by Feature A, even if Feature B is equally or more predictive.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.6, Ch.9](#); [Pandas_Zero_to_Master.ipynb Ch.7](#)